



DSHOT

DSHOT is the name coined for a new digital ESC protocol by Felix (KISS) who is working in collaboration with Boris and the rest of the betaflight team, and Steffen (BLHeli) is also playing his part and introducing this protocol to BLHeli_S.

Note: BLHeli stands for Brush Less Helicopters

Make no mistake (despite what some have said) although this protocol uses the PWM (Pulse Width Modulation) features of the micro-controllers it is in fact digital. Each pulse represents a single bit. The timing of that pulse, as in the duration, in relation to the overall period between pulses dictates whether the bit is ON or OFF i.e. a bit value of a ZERO or a ONE. The micro-controllers are simply using the direct memory access, and timer capability within micro-controllers to generate the necessary signals on a motor pin so as to minimise the CPU utilisation.

What is the difference to current protocols?

Non-digital PWM protocols, such as Multishot, Oneshot42, Oneshot125 etc., simply rely on the width of the pulse to indicate throttle position. This has a number of significant drawbacks. In those analogue protocols subtle timing variations are a phenomenon we all know as jitter. This is where the signal jumps around the desired point. The shorter the pulse width the higher the likelihood and impact of this jitter. We have all seen an example of jitter in the receiver tab of the respective xFlight configurators. Variations in the pulse width also result in an unknown zero throttle, and an unknown maximum throttle. It is this reason that calibration of ESCs is required when using PWM.

Pretty much all digital communications use timing and voltage levels against some form of reference (usually a ground) to indicate bit values. Serial (e.g. RS232, RS485 etc) uses the high (voltage > reference) to indicate a bit value of 1, and a low (voltage closely approximating reference or < reference) to represent a bit value of 0. In the case of serial each end of the communication needs to know the timing to utilise as consecutive bits in a 1 state are merely continuously highs (no pulses). The baud rate is therefore critical. In the case of DSHOT the timing is less critical. The reason for this is that the pulse width (duration) as a proportion of the total period denotes the bit value. This is very similar to the way in which WS2811/2 LED

Strips work. In fact aside from the different timings, it would be pretty much identical.

What is in a name?

For DSHOT600 the 600 signifies the bit-rate (in kilo-bits), so it can send 600 kilo-bits per second). For DSHOT300, and DSHOT150 it is essentially the same, but the bit-rate is slowed, for 300 (300 kilo-bits per second) it is 2x slower, for 150 (150 kilo-bits per second) it is 4 times slower. This means the timing above is simply x2 for 300, and x4 for 150. DSHOT300 and DSHOT150 were introduced to ensure that support for older less capable ESCs exists so flyers can get the benefit of digital accuracy.

DSHOT bucks the trend, the higher number is faster than the lower numbers. The oneshot protocols (including Multishot) have used increasingly lower numbers to signify faster speeds.

What are the technical details?

For DSHOT600 the timing output is as follows:

DSHOT-sequence

Bit length (total timing period) is 1.67 microseconds ($T_{0H} + T_{0L}$ or $T_{1H} + T_{1L}$). For a bit to be 0, the pulse width is 625 nanoseconds (T_{0H} – time the pulse is high for a bit value of ZERO) For a bit to be 1, the pulse width is 1250 nanoseconds (T_{1H} – time the pulse is high for a bit value of ONE) The reason for the difference in pulse length for bit 0 and bit 1 values is that it allows for considerable tolerance in determining the value. So these timings can be off slightly and the result will still be the same.

As with any protocol there needs to be a communication stream of bits, that will be interpreted for some outcome to occur. For the case of DSHOT the signal for a motor update consists of 16 bits (a frame). The first 11 bits are the actual throttle value. The next bit is to signal the ESC to provide a telemetry update (using a separate return channel), and the remaining 4 bits are a checksum.

DSHOT-output

The throttle value with 11 bits gives a resolution of 2048. It has been suggested the first values be reserved (possibly for startup tones or commands), so 0 means disarmed. So if 1 to 47 are reserved, and then 48-2047 is the throttle position – giving 2000 steps of resolution.

In the throttle value portion of the frame the most significant bits are first, so the first bit having a value of 1 means the throttle is at least half way, i.e. 1024, the second bit represents 512, the third 256 and so on until bit 11 represents the value of 1. So the bit sequence 1111111111 represents full throttle, and 10000000000 represents half throttle. The following image is a capture of DSHOT600 in action on a BluejayF4:

DSHOT-capture

There is normally a pause between frames of at least 2 microseconds to indicate a frame reset. A reset simply indicates the end of one frame and thus any future bits are the start of a new frame. With DShot occurring at the end of a PID loop this pause is actually considerably longer. If DShot were to be made to continuously output a signal then this delay would be required.

Is it the most efficient?

It should of course be noted that DShot is not the absolute most efficient protocol for representing bit values in a serial stream, but the reason for the approach taken rather than other options is to ensure accuracy in the very noisy environment found on a multi-rotor. That said it is certainly no slouch. It takes it to the current crop, with DSHOT600 being a mere 1 microsecond slower than Multishot at full throttle. Even DSHOT150 is on a par with Oneshot42. The following graph represents the full throttle timing of the current protocols:

DSHOT has no real concept of minimum and maximum durations for timing, it is a digital protocol after all. A disarmed throttle value of 0 takes the same amount of time to communicate as a full throttle value.

Will my ESCs or flight controller be supported?

Work is being done in KISS firmware and in Betaflight to support this new protocol, and so far testing is looking very promising. It is unlikely that F1 processors (Naze, CC3D etc) will be able to support it, due to the direct memory access requirements of the hardware. Those processors just don't have enough DMA channels available. F3, F4 and F7 all have more than enough. However inside those processors certain mappings exist – between timers, DMA and the actual pins – and so it is possible we will find a target where you won't be able to simply use motor outputs 1 to 4, but may have to move to motors 1, 2, 4 and 5 (as an example). Resource remapping in Betaflight 3.1 when released will make this task nice and easy (aside from any soldering if needed).

For ESCs the hardware requirements will mean BLHeli_s or better. There are also ARM based ESCs, such as KISS 24A RACE, that will support DShot or are able to be

upgraded to support it. Unfortunately the old ATMEL and slower SILABs based ESCs are unlikely to ever have the power nor timing hardware needed to support DSHOT. Either way it will mean a firmware upgrade, and in some cases this will mean soldering if the ESC does not have a boot loader that allows simplified firmware updates.

Ok – so what are the benefits?

The key benefit to digital is there is absolutely no increase in value through repeating the same information, more than once. Digital is simply accurate. There is no jitter (variations in throttle value created due to timing). If the signal is corrupted the ESC can detect it through the checksum. Many will have found motor updates of 32khz appear smoother. This is most likely to do with the repeating signal being "averaged" out effectively and thus effectively filtering (smoothing) the jitter. DShot eliminates this PWM jitter, and therefore motor updates need not occur anymore frequently than a motor can actually physically adjust – and make no mistake there is a physical limit as to how quick a motor can change its speed.

Another benefit is of course, no calibration required. In the digital world a zero is just that, a zero.

So really now it is just a case of rest in peace PWM, for we have a new friend, and that friend goes by the name of DSHOT!

Joe Lucid took it further and introduced bidirectional DShot

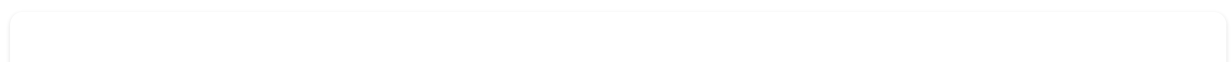
DSHOT bidir uses inverted signal levels (idle is 1). FC to ESC uses dshot frames but the lowest 4 bits hold the complement of the other nibbles xor'd together (normal dshot does not complement the xor sum). The ESC detects based on the inversion that telemetry packets have to be sent.

30us after receiving the dshot frame the esc responds with a telemetry frame. Logically the telemetry frame is a 16 bit value and the lowest 4 bits hold the complemented xor'd nibbles again.

The upper 12 bit contain the eperiod (1/erps) in the following bitwise encoding:

$$\begin{matrix} e & e & e & m & m & m & m & m & m & m & m & m & m & m & m & m \end{matrix}$$
 The 9 bit value M needs to be shifted left E times to get the period in micro seconds. This gives a range of 1 us to 65408 us. Which translates to a min e-frequency of 15.29 hz or for 14 pole motors 3.82 hz.

This 16 bit value is then GCR encoded to a 20 bit value by applying the following map nibble-wise:



```
0 -> 19
1 -> 1b
2 -> 12
3 -> 13
4 -> 1d
5 -> 15
6 -> 16
7 -> 17
8 -> 1a
9 -> 09
a -> 0a
b -> 0b
c -> 1e
d -> 0d
e -> 0e
f -> 0f
```

This creates a 20 bit value which has no more than two consecutive zeros. This value is mapped to a new 21 bit value by starting with a bit value of 0 and changing the bit value in the next bit if the current bit in the incoming value is a 1, but repeating the previous bit value otherwise. Example:

1 0 1 1 1 0 0 1 1 0 would become 0 1 1 0 1 0 0 0 1 0 0.

This 21 bit value is then sent uninverted at a bitrate of $5/4$ * the dshot bitrate. So DShot 3 uses 375 kbit, dshot 600 uses 750 kbit.

The esc needs to be ready after 40us + one dshot frame length to receive the next dshot packet.

See [https://en.wikipedia.org/wiki/Run-length_limited#GCR:_\(0,2\)_RLL](https://en.wikipedia.org/wiki/Run-length_limited#GCR:_(0,2)_RLL) for more details on the GCR encoding.

See <https://github.com/betaflight/betaflight/pull/8554>

Youtube

 Joshua Bardwell

 Ivan Efimov

Community

 Discord

 IntoFPV

Links

 Oscar Liang

 VitroidFPV

More

 Blog

 GitHub



Copyright © 2025 All rights reserved Team Betaflight

Built with Docusaurus

This site is powered by Netlify

made with ❤️ by **VitroidFPV** and **un!t**

